Diseño preliminar: Biblioteca escolar con ejemplares irregulares

Día 1 — Documento de diseño preliminar

1. Objetivo del proyecto

Desarrollar una aplicación para una biblioteca escolar que permita administrar títulos con distintas cantidades de ejemplares. El sistema deberá facilitar la consulta de disponibilidad, el registro de préstamos, el ordenamiento por préstamos históricos y la búsqueda de libros por ISBN.

2. Problema de los ejemplares irregulares

Los títulos no tienen necesariamente la misma cantidad de ejemplares. Por ejemplo, un libro puede tener una copia y otro puede tener cinco. La estructura elegida debe representar esas diferencias sin reservar espacios vacíos para completar filas.

3. Comparación de estructuras candidatas

Arreglo escalonado (int[][]): cada fila puede tener una longitud diferente. Ventajas: representa filas de tamaños distintos y utiliza arreglos. Desventajas: los datos suelen identificarse mediante índices y resulta menos claro relacionar cada número con su significado; además, cambiar el tamaño de una fila requiere crear otro arreglo.

4. Comparación de List

List: permite agregar o quitar filas y cada arreglo interno puede tener diferente longitud. Ventajas: flexible y admite cantidades variables de elementos por título. Desventajas: se depende de posiciones numéricas para saber qué dato representa cada índice; el código puede ser difícil de mantener y validar.

5. Comparación de List

List: almacena objetos de una clase Titulo con propiedades identificadas por nombre, por ejemplo Nombre, ISBN, EjemplaresTotales, EjemplaresDisponibles y PrestamosHistoricos. Ventajas: los datos de cada libro permanecen agrupados, el código es legible y resulta sencillo modificar la disponibilidad o los préstamos. Desventajas: requiere definir una clase y comprender objetos y propiedades.

6. Estructura seleccionada y justificación

Se propone utilizar List. Esta estructura permite que cada título tenga su propia cantidad de ejemplares, sin celdas vacías. También facilita recorrer el catálogo, agregar títulos y actualizar sus propiedades cuando se registra un préstamo. Aunque requiere una clase, ofrece una representación más clara que guardar los datos en posiciones numéricas.

7. Plan de trabajo posterior

Día 2: crear el modelo Titulo y cargar ocho títulos de prueba. Día 3: implementar disponibilidad y préstamos. Día 4: programar Insertion Sort y probar el desempate alfabético. Día 5: implementar búsqueda binaria por ISBN y secuencial por título. Día 6: integrar y probar. Día 7: comparar el número de comparaciones con Array.BinarySearch. Día 8: preparar informe, anexos y sustentación.

8. Conclusión preliminar

List es la alternativa recomendada para este proyecto porque representa cada libro como una unidad con propiedades claras y permite manejar cantidades irregulares de ejemplares sin espacios vacíos. La decisión se revisará al implementar y probar las funcionalidades.

Integrantes: [Escribir nombres del equipo]

Fecha: [Escribir fecha]